Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação

Implementação e comparação entre métodos heurísticos para a resolução do problema de corte de itens irregulares em faixa

Nina Cunha Pinheiro

Trabalho de Conclusão de Curso
Relatórios finais de Iniciação Científica (PUB): set/2023–ago/2024 e set/2024–ago/2025

Orientadora: Profa. Dra. Marina Andretta

São Carlos
2026

1 Introdução

Otimização é uma área da matemática que busca resolver problemas de alta complexidade almejando encontrar a melhor solução que formos capazes para aquele problema, chamada solução ótima. A modelagem de problemas, por sua vez, pretende traduzir problemas do mundo real para a linguagem matemática, para conseguirmos futuramente analisá-los e, por exemplo, otimizá-los.

Problemas de corte e empacotamento são problemas otimização que buscam encaixar itens em um dado recipiente, de forma a minimizar desperdício de espaço a ser ocupado. Eles podem ser aplicações de problemas da vida real como corte de tecidos na indústria têxtil, corte de itens na automobilística, na indústria de calçados e até na disposição de itens em um contêiner para transporte.

Nesse trabalho estudamos o problema de corte de itens irregulares em faixa, que consiste em posicionar itens irregulares em uma faixa, sem sobreposição, de forma a minimizar o comprimento usado da faixa. Estudamos também ferramentas para resolver esse problema, como NFPs, IFPs, a heurística construtiva *Bottom-Left* e a meta-heurística *BRKGA*.

adicionar referências de trabalhos relacionados

Na Seção 2 mostramos o problema de corte de itens irregulares em faixa e explicamos principalmente as ferramentas utilizadas para garantir que os itens não se sobreponham e permaneçam no interior da faixa. Na Seção 3 detalhamos a heurística construtiva *Bottom-Left* implementada para resolvermos o problema apresentado na Seção 2, juntamente com alguns resultados de seus testes na Seção 3.1. Na Seção 4 detalhamos o funcionamento e implementação da meta-heurística *BRKGA* para a resolução do mesmo problema e analisamos os resultados obtidos na Seção 4.1. Na Seção 5, comparamos os resultados dos experimentos numéricos de ambas as heurísticas e por fim, na Seção 6, apontamos as considerações finais e possibilidades de trabalhos futuros.

2 Formulação de um problema de corte de itens irregulares

Problemas de corte ou empacotamento são aqueles em que se deve organizar uma série de objetos em um recipiente de forma a minimizar desperdício. Como forma de resolução, inicialmente o problema é interpretado e modelado em linguagem matemática. Em seguida, pode ser resolvido de forma exata, encontrando uma solução ótima, ou aproximada, com uma solução boa o suficiente para as especificações, utilizada para diminuir os gastos com tempo e capacidade de processamento provindas da primeira.

Ao longo desse trabalho queremos encontrar soluções possíveis e relativamente boas para o problema que consiste em posicionar m itens irregulares (representados por polígonos), cada um com determinado número de cópias (quantas vezes um mesmo item deve ser alocado), em uma faixa de comprimento infinito (um retângulo com um lado de tamanho fixo e o outro que se estende o quanto for necessário), sem sobreposição, de forma a minimizar o comprimento da região da faixa utilizada. Observe na Figura 1 um exemplo de uma solução desse problema de corte, mais detalhes são apresentados na Seção 3.1.

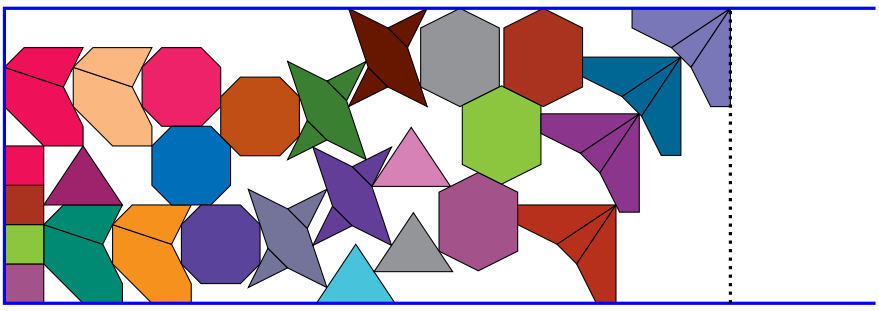


Figura 1: Exemplo de um problema de corte de itens irregulares em faixa. Em azul temos a faixa infinita e a linha vertical pontilhada é o comprimento alcançado.

O problema de corte em faixa reflete o dilema de muitas indústrias, como a têxtil, ao recortar modelos de roupa de um rolo de tecido, ou a automobilística, recortando itens de carro em uma chapa de metal. Caso um polígono sobreponha outro os itens recortados se tornam inúteis quando o problema é aplicado à vida real, como nestes casos industriais, que é um dos motivos pelo qual não pode haver sobreposição de itens.

Dessa forma, é imprescindível estar atento se na alocação final não haja dois itens que estão ocupando um mesmo espaço e se todos se encontram no interior do recipiente - a solução seria inutilizável caso contrário. Em desafios bidimensionais, em que os itens são polígonos irregulares, com os quais vamos trabalhar, para averiguar que tais necessidades são atendidas utilizamos os métodos *Inner-Fit Polygon* (IFP) e *No-Fit Polygon* (NFP).

Existem outros métodos para a verificação, como os encontrados em [1]. Para usar tanto o IFP quanto o NFP, utilizamos algum ponto P do item para localizá-lo, dado como ponto de referência e cuja posição final indica quanto os outros pontos do item serão deslocados.

O IFP de um item X é o polígono dado por toda área do recipiente onde o ponto de referência de X pode ser colocado de forma que este item esteja contido por completo no recipiente. No problema com o qual estamos trabalhando o recipiente é a faixa de comprimento infinito, um retângulo de altura A determinada e comprimento C a ser minimizado, com o ponto inferior esquerdo posicionado em $(0, 0)$.

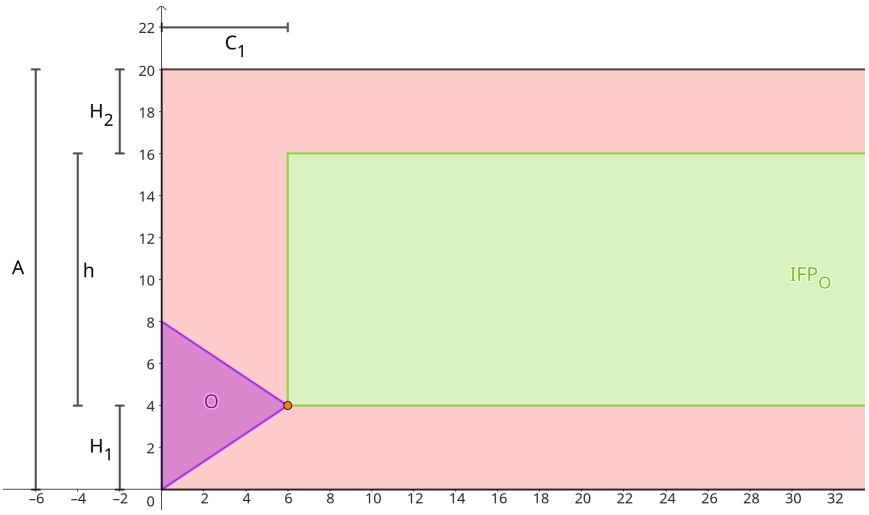


Figura 2: A área em verde é o IFP do polígono O.

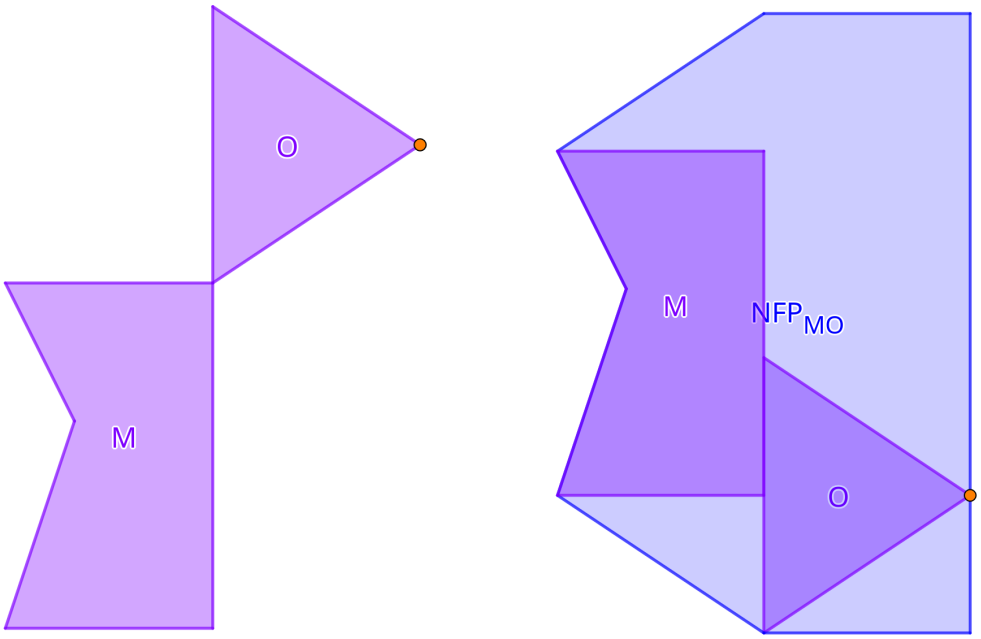
No exemplo da Figura 2, o retângulo delimitado em preto é a faixa e queremos posicionar O em seu interior. O IFP_O está representado na figura pela área em verde e o ponto de referência P é o em destaque laranja. Na figura, H_1 e H_2 representam, respectivamente, as distâncias entre P e o ponto mais baixo de O e entre P e o ponto mais alto de O, e h é a distância vertical restante para posicionar P. Além disso, P só pode ser colocado horizontalmente à direita de C_1, distância entre P e o ponto mais à esquerda de O. No desenho, O estaria alocado no ponto mais à esquerda e abaixo possível.

Agora que analisamos a relação de O com o recipiente, considere que queremos entender a interação entre os polígonos M e O presentes na Figura 3a, para garantir que um não irá sobrepor o outro. Uma forma de fazer isso é manter M parado e, garantindo que O encoste nele o tempo todo, transladar O ao seu redor, trançando o caminho feito pelo ponto de referência de O. O polígono formado por esse caminho é o que chamamos de *No-Fit Polygon (NFP)* e seu interior indica onde haverá sobreposição entre os dois, se colocarmos o ponto de referência de O. Caso o ponto de referência esteja no exterior do NFP, O e M não se encostarão.

Ao longo desse trabalho, chamamos de NFP_{MO} aquele em que M está fixo e O translada ao seu redor, como observado na Figura 3b. É importante notar que seu desenho poderá ser diferente do NFP_{OM}, o NFP gerado se alterarmos qual polígono translada e qual está fixo.

Existem diversas maneiras de construir NFPs, como pode ser visto em [2]. A mais intuitiva é a que acabamos de apresentar, transladando um em torno do outro, garantindo que estejam encostados e traçando a fronteira a partir do ponto de referência do polígono transladado. Vale destacar que, no nosso problema, a orientação dos polígonos é fixa, ou seja, eles não rotacionam.

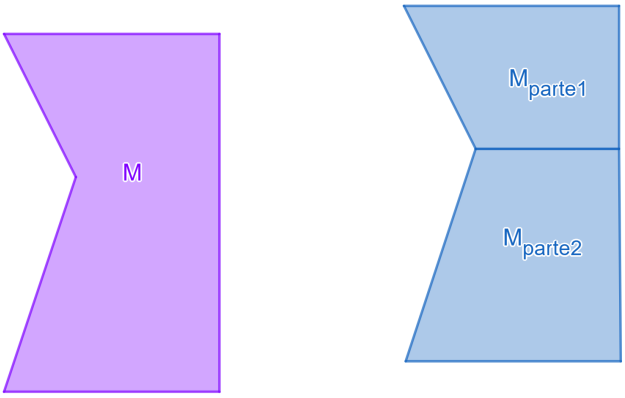
Os algoritmos para calcular NFPs de polígonos convexos são mais simples, enquanto os para cálculo de não convexos são bem mais complicados. Portanto, quando um polígono (ou ambos) não é convexo, iremos decompô-lo em polígonos convexos menores (que chamamos de partes), para facilitar os cálculos, e o NFP será montado a partir da translação de cada parte separadamente. O NFP final de itens não convexos é constituído de partes que se sobrepõem e é importante citar que o item apenas estará em uma posição válida se ele não estiver no interior de nenhuma das partes.



(a) O transladando M.

(b) O transladando M - NFP_{MO}.

Figura 3: O polígono O sendo transladado ao redor de M e o caminho feito pelo seu ponto de referência (em laranja) resultando no NFP_{MO}.

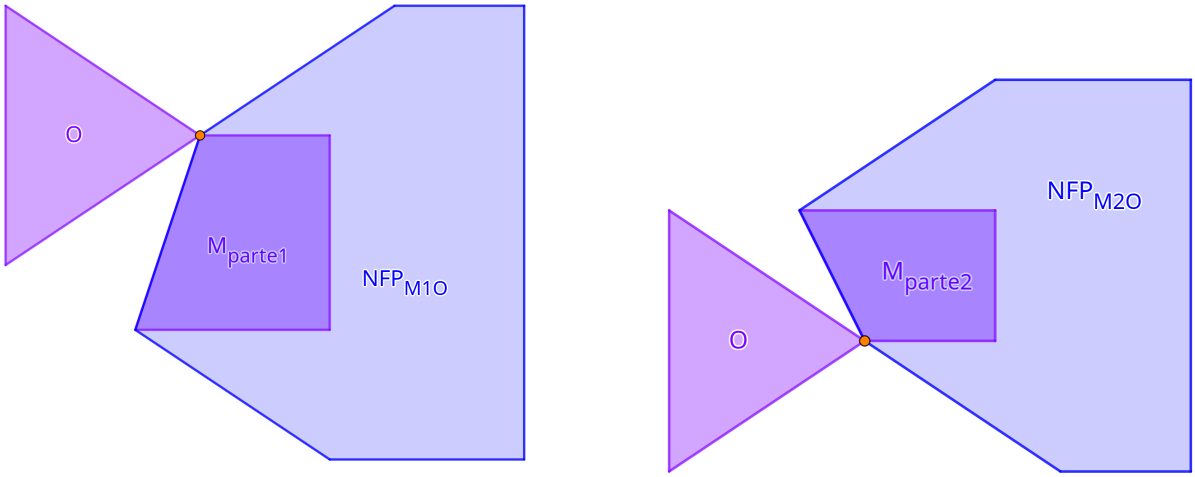


(a) Polígono M não convexo.

(b) Partes 1 e 2 convexas de M.

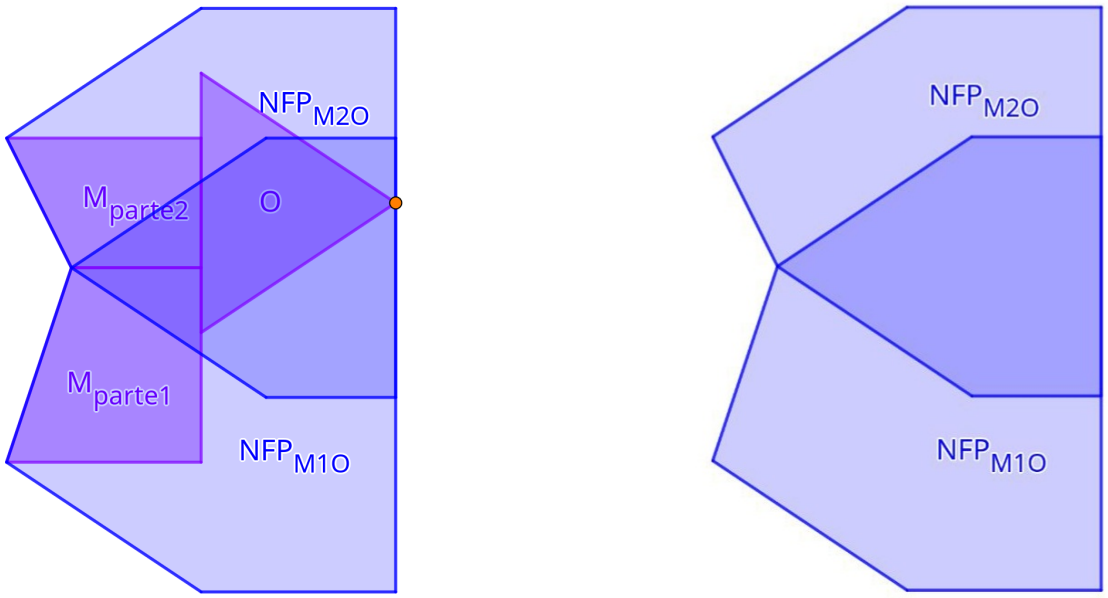
Figura 4: Polígono M separado em duas partes por não ser convexo.

Perceba que o polígono M apresentado anteriormente não é convexo. Para construir o NFP_{MO}, podemos decompô-lo ele em dois polígonos convexos, como possível observar nas Figuras 4a e 4b, e vamos construir o NFP de O com relação às partes 1 e 2 do polígono M. Nas Figuras 5a e 5b temos O transladando cada parte de M. Observe que deve ser feito um ajuste final no posicionamento de cada parte do NFP ao juntá-las, tendo em mente qual a posição de uma parte em relação à outra. Na Figura 6a temos a junção das partes e na Figura 6b temos o NFP_{MO} por essa montagem, cujo contorno exterior podemos observar ser exatamente o mesmo da Figura 3b.



(a) O transladando M_{parte1} - NFP_{M_1O}. (b) O transladando M_{parte2} - NFP_{M_2O}.

Figura 5: Montagem das duas partes do NFP_{MO}.



(a) Partes de M, O e as partes de NFP_{MO}. (b) NFP a partir das duas partes.

Figura 6: Junção das duas partes do NFP_{MO}.

Na próxima seção vamos colocar todas as ferramentas apresentadas em uso para implementar uma heurística que resolva o problema de cortar itens irregulares como esses, convexos e não convexos, em faixa.

3 Heurística construtiva para resolver o problema de corte de itens irregulares

Uma heurística é uma forma de encontrar uma solução suficientemente boa para um problema de otimização, diminuindo os gastos computacionais e de tempo que seriam gastos para encontrar uma solução ótima. Dada

uma lista de polígonos ordenados segundo algum critério, a heurística *Bottom-Left*, estudada por meio de [2], consiste em alocar um polígono de cada vez no ponto mais à esquerda possível da faixa e, caso haja mais de um ponto, mais abaixo. Dessa forma, é uma heurística simples para problemas de empacotamento, que iremos aplicar para resolver o problema citado na Seção 2.

Utilizando como exemplo a Figura 7, temos os polígonos já conhecidos M e O e um quadrado N de lado 6, na faixa de altura $A = 20$. A ordem de posicionamento escolhida é alfabética e, dado que M e N já estão situados de acordo com a Figura 7 e o IFP de O é o mesmo da Figura 2, vamos alocar O utilizando o NFP_{MO}, o NFP_{NO} e o IFP_O.

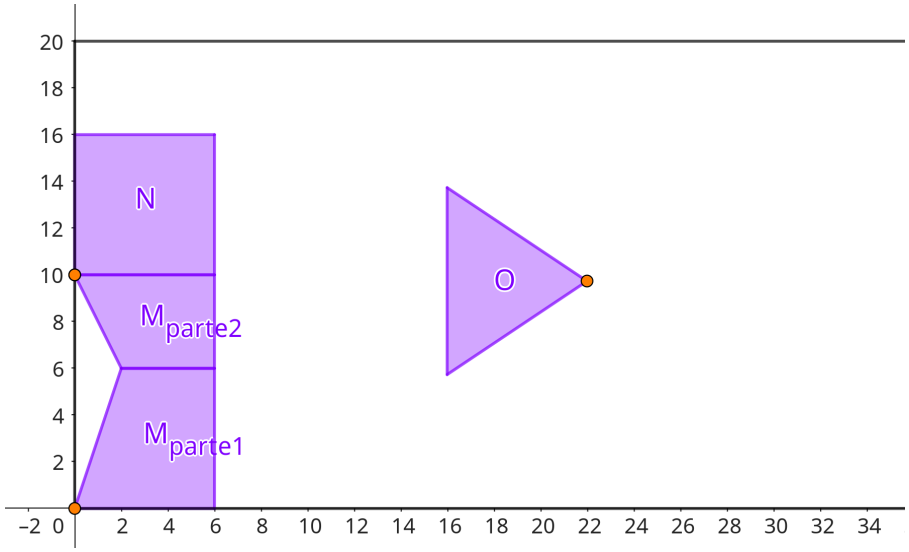


Figura 7: Polígonos M e N alocados na faixa e O aguardando alocação.

Uma maneira de encontrar as possíveis posições para alocar o polígono O é calcular todas interseções das arestas entre NFP_{MO}, NFP_{NO} e IFP_O e remover as que não forem viáveis. Note que isso é prático pois, ao alocarmos um item, sempre é possível encaixá-lo melhor ao garantir que ele esteja encostado ao menos em duas arestas (sejam elas de itens ou da fronteira do recipiente).

Observe na Figura 8 os NFPs deslocados baseados na alocação do seu respectivo polígono base e na Figura 9 todas as intersecções. Os pontos em vermelho são aqueles que estão na área proibida de alguma das restrições, ou seja, dentro de algum NFP ou no exterior do IFP. Ademais, podemos ver que entre os cinco pontos permitidos, que estão em verde, não existe um mais à esquerda (dado que todos tem mesma abscissa) e o mais abaixo é aquele onde O deve ser posicionado (ponto P_5).

Na Figura 10 temos então o polígono O em sua posição final de acordo com a heurística *Bottom-Left*, em P_5.

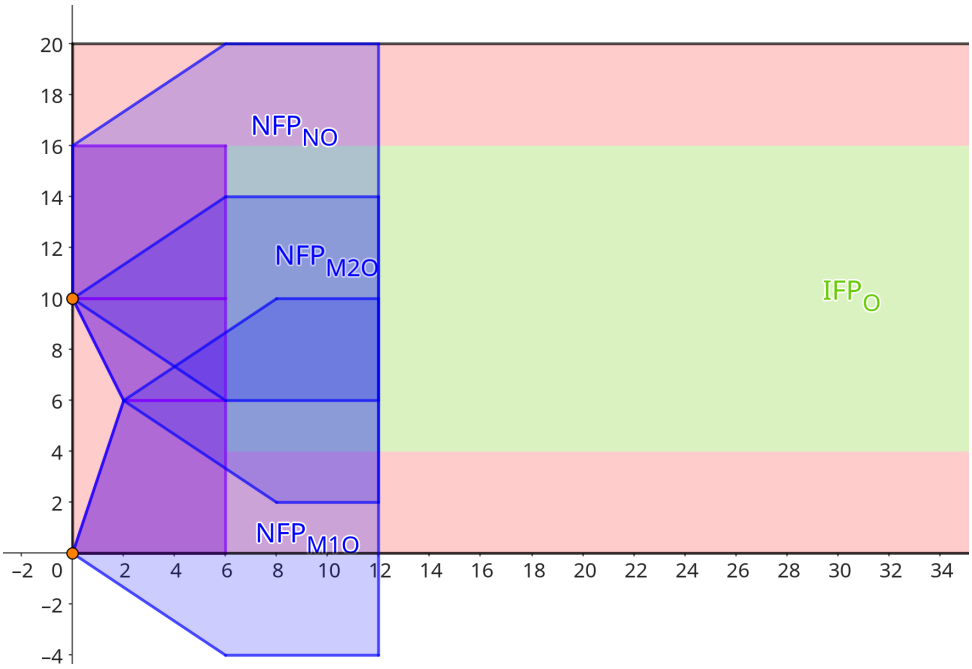


Figura 8: Polígonos M e N, com os respectivos NFPs em relação a O (note que os NFPs estão posicionados para encaixar perfeitamente com o polígono fixo referente a ele). As intersecções gerarão os possíveis pontos para alocar o ponto de referência de O.

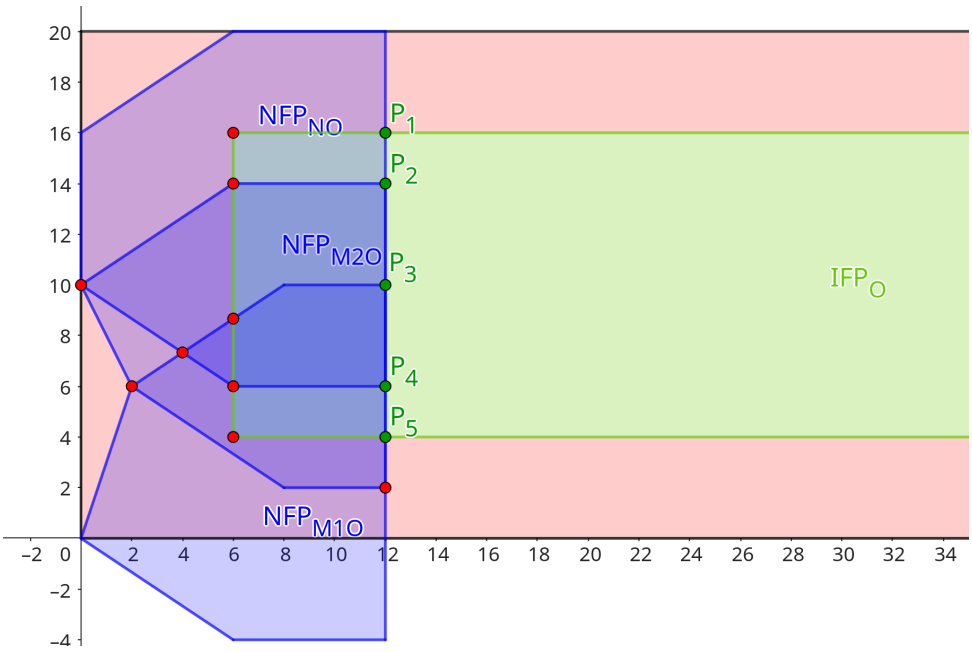


Figura 9: NFP_{M_1O}, NFP_{M_2O}, NFP_{NO} e os pontos de intersecção entre todos e o IFP_O, onde os pontos em vermelho são posições proibidas e os em verde são posições viáveis.

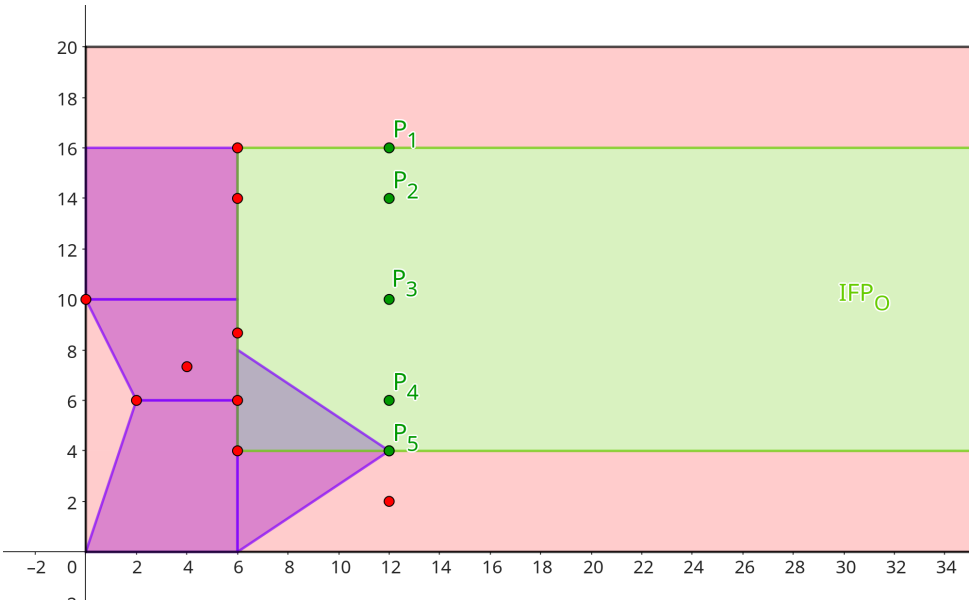


Figura 10: Todos os pontos de intersecção de O e a posição final onde foi alocado (P_5).

Agora que temos todas as ferramentas de alocação bem sedimentadas, vamos para a resolução em si. Para resolver o problema de corte de itens irregulares em faixa foi implementado um código em linguagem C com a heurística *Bottom-Left*. O programa lê em um arquivo "general_data.txt" a altura A da faixa, o número m de itens e m linhas seguintes contendo o número de cópias de cada item. Em seguida, é lida uma lista de m arquivos "pieceX.txt" que diz o número de partes do item X, representado por um número de 1 a m, e lista os vértices em sentido anti-horário para cada parte, considerando que o primeiro ponto da primeira parte é $(0, 0)$, que é o ponto de referência de X.

Por último, o programa lê m^2 arquivos "NFP_i_0_j_0.txt" com as partes e vértices de cada NFP, construídos pensando no ponto de referência do item i em $(0, 0)$ e o item j transladando ao seu redor, ambos com rotação de 0 graus.

A estrutura de dados consiste em armazenar em uma estrutura referente a um item X seu polígono, seus pontos extremos, o número de cópias, a lista de NFP_{XJ} (onde $J = 1, \ldots, m$), o vértice do ponto de referência em sua alocação final para cada cópia e uma variável auxiliar para numerar quantas cópias já foram alocadas.

Ainda no terminal, antes de executar o programa, é usado um *makefile* para compilar o programa e gerar um executável, especificando nele quais as instâncias a serem resolvidas e qual ordenação será utilizada para a *Bottom-Left*. As ordenações foram separadas em duas categorias que são combinadas na hora de aplicar: método e sentido da ordenação. O método é dado por um número de 1 a 6 que dita qual valor a ser observado para ordenar o itens, as opções são as seguintes:

1. ordenação baseada na área do item;
2. ordenação baseada na área do menor retângulo que contém o item;
3. ordenação baseada no comprimento de cada item (diferença entre pontos extremos em relação ao eixo X);
4. ordenação baseada na altura de cada item (diferença entre pontos extremos em relação ao eixo Y);
5. ordenação baseada na divisão da área do item pela área do menor retângulo que o contém;
6. ordenação baseada na divisão entre o comprimento (diferença entre pontos extremos em relação ao eixo X) e altura (diferença entre pontos extremos em relação ao eixo Y) de cada item.

Em relação ao sentido da ordenação temos apenas dois valores possíveis: 1 ordena o índice com valores dos cálculos do método de forma decrescente e 2 de forma crescente. Então, após a leitura de dados, o programa passa pelas seguintes etapas:

1. Ordena todos os itens (incluindo as cópias) utilizando a opção escolhida;
2. Para cada item a, a partir da ordem do Passo 1:

(a) Calcula o IFP_a;

(b) Verifica se o vértice inferior esquerdo do IFP_a é válido (ou seja, se não está no interior de nenhum NFP_{ai}, com i os itens já alocados na faixa);

(c) Em caso positivo, define M (que guarda o melhor ponto para alocar um item a a cada iteração) como esse e pula para o Passo 2.(e);

(d) Caso contrário:

i. Defina M como um ponto de coordenadas $(g, 0)$, em que g é um valor muito grande que não pretendemos alcançar no comprimento da faixa (calculado, por exemplo, somando o comprimento de todos os itens a serem alocados);

ii. Calcula todas as intersecções duas a duas entre os NFP_{ai} e NFP_{aj}, com i e j os itens já alocados na faixa;

iii. Calcula todas as intersecções entre os NFP_{ai}, com i os itens já alocados na faixa, com o IFP_a;

iv. Para cada intersecção I encontrada, se a coordenada x de I é menor do que a coordenada x de M (ou se a coordenada x de I é igual à coordenada x de M e a coordenada y de I é menor do que a coordenada y de M), atualiza M com o valor de I;

(e) Aloca o item a no ponto M da faixa.

Por fim o programa imprime um arquivo em Latex utilizando a ferramenta *Tikz* do leiaute correspondente, em que o recipiente é representado em azul, o comprimento obtido é a linha vertical pontilhada e os itens são coloridos (itens não convexos são representados por suas partes convexas conectadas, de mesma cor). Imprime também as informações de comprimento mínimo alcançado e tempo de execução num único arquivo de texto para facilitar sua disposição em forma de tabela (como a da Seção 3.1).

3.1 Experimentos numéricos

Para que fosse feito o teste do código, implementado em linguagem C, executamos a heurística *Bottom-Left* com todas as combinações de método e sentido de ordenação apresentadas na Seção 3, totalizando doze variantes da heurística, chamadas de BL_{MS}, em que M representa o método e S o sentido da ordenação. As variações foram aplicadas em vinte e nove instâncias clássicas da literatura[1], as presentes nas tabelas 1 e 2.

Os experimentos foram executados em um computador de 8GB de memória, processador Intel® Core™ i5-8250U x 8 e Sistema Operacional Ubuntu 24.04.2 LTS. Tanto o código em C quanto os arquivos base das instâncias e todos os leiautes obtidos ao longo deste trabalho de Iniciação Científica podem ser acessados em `https://sites.icmc.usp.br/andretta/pinheiro-2025/`.

Os resultados obtidos foram divididos em duas tabelas 1 e 2. A coluna "Instância" contém o nome de cada instância utilizada, a "Itens" retrata o número de itens da instância (incluindo as cópias de cada item). Em seguida, cada grupo de duas colunas representa o resultado da execução de uma combinação BL_{MS}, em que $M = \{1, 2, \ldots, 6\}$ e $S = \{1, 2\}$. A coluna "Comp" representa o comprimento obtido para a instância pela heurística BL_{MS} correspondente e a coluna "T(s)" apresenta o tempo em segundos que a heurística levou para resolver cada instância. O valor em destaque em cada linha é o menor valor entre todos os resultados para aquela instância. Na Tabela 1, temos as 6 variações da heurística que apresentaram as maiores quantidades de instâncias para as quais se obteve melhores comprimentos, ordenadas da que obteve maior quantidade de melhores resultados para a que obteve menor quantidade. Na Tabela 2 estão as variantes restantes da heurística.

Como podemos observar na Tabela 1, a ordenação dos itens de forma decrescente baseada no comprimento de cada um (BL_{31}) obteve o maior número de bons resultados (12 de 29 instâncias) e, assim como todas as variações, num tempo de execução muito pequeno para todas as instâncias. Para entender a disparidade existente entre as variações da heurística aplicadas a uma mesma instância, dividimos o menor valor de comprimento obtido para cada uma pelo maior. Assim, observamos a maior diferença de valores na instância *poly5A*, com razão $0,895$, e a menor na instância *poly1A*, com razão $0,715$. Ou seja, notamos que como nenhuma proporção resultou em 1, variar a ordenação para aplicar a heurística gera resultados diferentes em todos os casos. Quando comparamos somente os resultados das heurísticas BL_{31}, BL_{21}, BL_{52} e BL_{11}, as quatro com melhores resultados, a posição entre qual tem maior número de melhores resultados não foi alterada.

[1] `https://www.euro-online.org/websites/esicup/data-sets`, acessado em 20/02/2025.

Tabela 1: Resultados obtidos pelas 6 variações da heurística *Bottom-Left* que obtiveram maior quantidade de menores valores de solução.

		BL_{31}		BL_{21}		BL_{52}		BL_{11}		BL_{12}		BL_{22}	
Instância	Itens	Comp	T (s)	Comp	T (s)	Comp	T (s)	Comp	T (s)	Comp	T (s)	Comp	T (s)
albano	24	11376,46	0,42	11818,58	0,29	13525,12	0,36	11818,58	0,37	14679,58	0,42	13959,22	0,43
blaze1	28	28,67	0,22	31,20	0,20	34,34	0,21	30,35	0,28	32,14	0,25	36,67	0,33
blaze2	16	21,50	0,08	22,10	0,06	25,29	0,04	21,50	0,09	22,50	0,05	24,93	0,08
dagli	30	75,47	0,57	73,33	0,39	89,92	0,23	67,69	0,43	86,35	0,18	78,40	0,19
dighe1	16	146,96	0,00	167,06	0,00	165,75	0,00	163,26	0,00	203,00	0,01	187,03	0,01
dighe2	10	145,56	0,00	163,23	0,00	165,16	0,00	154,30	0,00	182,78	0,00	166,79	0,00
fu	12	42,25	0,00	36,46	0,00	43,46	0,00	42,00	0,00	46,25	0,00	42,00	0,00
jakobs1	25	14,00	0,10	13,00	0,08	17,00	0,06	13,00	0,09	16,00	0,06	17,75	0,07
jakobs2	25	30,00	0,21	30,00	0,17	32,00	0,17	29,60	0,16	34,50	0,09	35,50	0,10
mao	20	2119,63	0,53	2289,22	0,46	2509,27	0,46	2305,08	0,46	2744,35	0,90	2721,25	0,80
marques	24	89,60	0,32	87,00	0,28	96,00	0,26	93,10	0,33	100,10	0,29	104,10	0,35
poly1A	15	18,10	0,01	20,18	0,01	22,56	0,01	22,05	0,01	23,10	0,01	25,29	0,01
poly2A	30	33,33	0,10	32,31	0,09	40,14	0,09	36,11	0,11	38,82	0,09	42,84	0,10
poly2b	30	35,25	0,12	35,96	0,11	41,01	0,11	37,07	0,13	44,34	0,12	42,82	0,13
poly3A	45	50,08	0,39	52,28	0,35	56,83	0,31	53,63	0,38	57,11	0,32	59,87	0,38
poly3b	45	45,68	0,45	45,29	0,35	51,14	0,34	48,42	0,46	55,76	0,40	51,36	0,45
poly4A	60	66,52	0,86	66,96	0,82	68,35	0,73	69,11	0,95	71,11	0,74	75,25	0,96
poly4b	60	55,97	0,91	56,99	0,85	61,77	0,82	60,25	0,96	66,49	0,94	67,83	0,96
poly5A	75	80,06	1,74	80,39	1,55	85,44	1,48	82,98	1,95	89,37	1,60	88,19	1,90
poly5b	75	69,28	1,81	67,52	1,50	70,99	1,52	71,56	1,85	79,25	1,71	74,98	1,90
shapes0	43	67,00	7,26	68,50	5,92	66,00	4,59	68,50	6,04	74,00	5,01	74,00	5,22
shapes1	43	67,00	6,83	68,50	5,89	66,00	4,60	68,50	5,68	74,00	5,05	74,00	5,18
shirts	99	68,56	7,76	68,04	7,71	67,05	7,96	68,04	7,63	79,85	12,93	79,85	13,09
three	3	7,00	0,00	7,00	0,00	7,00	0,00	6,67	0,00	6,20	0,00	6,67	0,00
threep2	6	10,00	0,00	10,00	0,00	10,00	0,00	10,67	0,00	10,67	0,00	10,67	0,00
threep2w9	6	9,27	0,00	9,27	0,00	9,27	0,00	9,67	0,00	8,60	0,00	9,00	0,00
threep3	9	16,33	0,00	16,33	0,00	16,33	0,00	16,33	0,00	16,50	0,00	15,73	0,00
threep3w9	9	14,00	0,00	14,00	0,00	14,00	0,00	12,17	0,00	13,30	0,00	11,60	0,00
trousers	64	281,21	1,63	283,60	1,98	305,89	2,01	283,60	1,90	345,93	2,96	365,00	2,89

Tabela 2: Resultados obtidos pelas outras 6 variações da heurística *Bottom-Left*, que obtiveram menor quantidade de menores valores de solução.

		BL_{41}		BL_{42}		BL_{32}		BL_{61}		BL_{51}		BL_{62}	
Instância	Itens	Comp	T (s)	Comp	T (s)	Comp	T (s)	Comp	T (s)	Comp	T (s)	Comp	T (s)
albano	24	12525,18	0,35	13393,84	0,48	12099,60	0,39	13337,91	0,41	12816,09	0,40	12753,27	0,43
blaze1	28	29,75	0,25	36,89	0,27	36,92	0,30	32,52	0,37	33,75	0,32	31,13	0,26
blaze2	16	22,10	0,06	24,93	0,07	21,50	0,08	24,93	0,08	24,70	0,08	22,10	0,07
dagli	30	79,77	0,36	75,71	0,19	82,06	0,17	87,49	0,37	72,49	0,52	78,71	0,30
dighe1	16	147,11	0,00	173,48	0,00	206,93	0,01	177,55	0,01	187,75	0,00	172,14	0,00
dighe2	10	170,53	0,00	184,74	0,00	161,33	0,00	178,23	0,00	156,36	0,00	190,73	0,00
fu	12	43,67	0,00	45,11	0,00	44,50	0,00	38,00	0,00	38,00	0,00	40,00	0,00
jakobs1	25	15,00	0,08	17,00	0,06	17,75	0,05	15,00	0,08	14,00	0,07	15,00	0,06
jakobs2	25	29,00	0,14	32,80	0,10	33,35	0,10	31,25	0,09	32,20	0,13	32,12	0,10
mao	20	2492,53	0,43	2509,62	0,76	2536,04	0,85	2562,07	0,61	2404,44	0,76	2665,23	0,59
marques	24	91,00	0,36	109,56	0,31	111,03	0,33	96,82	0,26	103,33	0,30	107,94	0,34
poly1A	15	19,94	0,01	19,53	0,01	25,31	0,01	19,08	0,01	21,61	0,02	21,42	0,01
poly2A	30	35,41	0,10	37,16	0,11	39,40	0,10	33,34	0,11	36,67	0,16	34,28	0,10
poly2b	30	36,45	0,13	40,64	0,12	42,81	0,13	37,64	0,13	36,46	0,16	40,66	0,13
poly3A	45	51,25	0,33	53,31	0,44	57,83	0,38	49,20	0,40	52,99	0,53	52,69	0,31
poly3b	45	50,15	0,39	52,23	0,45	51,33	0,42	45,87	0,46	50,57	0,50	52,15	0,37
poly4A	60	66,14	0,79	72,48	1,18	73,13	1,08	66,21	1,03	71,01	1,35	67,24	0,74
poly4b	60	63,00	0,97	63,94	0,99	64,84	0,98	57,35	1,60	59,73	1,14	61,63	0,87
poly5A	75	82,74	1,55	86,55	2,15	89,43	2,00	84,28	1,89	84,82	2,84	82,66	1,63
poly5b	75	71,78	1,76	77,18	1,92	77,13	1,89	68,97	2,51	72,43	2,22	75,56	1,71
shapes0	43	77,00	6,46	71,00	6,47	68,50	6,12	67,00	7,69	70,00	7,72	79,00	6,32
shapes1	43	77,00	6,51	71,00	6,59	68,50	5,98	67,00	7,40	70,00	8,26	79,00	6,40
shirts	99	69,08	8,57	80,53	17,90	77,43	10,40	74,77	15,92	78,05	11,45	67,71	10,22
three	3	6,60	0,00	6,67	0,00	6,67	0,00	6,40	0,00	6,67	0,00	6,60	0,00
threep2	6	12,00	0,00	10,67	0,00	10,67	0,00	12,33	0,00	10,67	0,00	12,00	0,00
threep2w9	6	10,50	0,00	9,00	0,00	9,67	0,00	9,25	0,00	9,67	0,00	10,50	0,00
threep3	9	18,00	0,00	15,73	0,00	16,33	0,00	17,33	0,00	16,33	0,00	18,00	0,00
threep3w9	9	13,00	0,00	11,60	0,00	12,17	0,00	13,00	0,00	12,17	0,00	13,00	0,00
trousers	64	287,00	2,02	348,76	3,34	342,72	4,32	284,23	2,40	312,27	3,46	322,40	3,08

4 Meta-heurística para resolver o problema de corte de itens irregulares

Como forma de tentar encontrar soluções mais próximas da ótima, estudamos a partir dos artigos [4] e [5], o algoritmo genético *Biased Random-Key Genetic Algorithm (BRKGA)*, com a intenção de implementar esta meta-heurística para resolver o problema tratado neste trabalho. Como o próprio nome diz, esse algoritmo se inspira em genética e conceitos do ramo da biologia. Os principais elementos desse algoritmo são:

- As **chaves**, cada uma um valor $k \in [0, 1]$;
- O **cromossomo**, uma sequência de m chaves atreladas a alguma característica específica do problema a ser resolvido, que também pode ser chamado de **indivíduo**;
- A função ***fitness***, que devolve um valor f para cada indivíduo, utilizado para comparar indivíduos e analisar qual é melhor;
- A **população**, composta por I indivíduos;
- A **elite**, uma parcela da população composta de E indivíduos, aqueles com o melhor valor *fitness*;
- O **cruzamento**, um processo que a partir do cromossomo de dois indivíduos "pais" gera um novo indivíduo i, escolhendo aleatoriamente para cada item x de i uma chave advinda de um dos pais para atrelar a x. Dado que esse algoritmo genético é enviesado, como indicado pelo próprio nome, temos duas especificações:
  - Um dos pais escolhidos é proveniente da elite, com o objetivo de obter características melhores, e o outro da não-elite, para possivelmente diversificar os resultados;
  - Existe um viés fixo b que, para cada chave de i, indica qual a chance de que ela seja proveniente do pai elite, enquanto $1 - b$ é a chance de que origine do não-elite.
- A **mutação**, processo de criação de novos indivíduos, a partir da geração de novas chaves aleatórias.
- As **gerações**, utilizadas criar uma nova população a partir da geração anterior, usando a elite, cruzamentos e mutações.

Dessa forma, o algoritmo gera chaves para cada indivíduo, montando a população com I desses, ordena os cromossomos de cada indivíduo, geralmente de forma crescente, e chama a função *fitness*, obtendo o valor f de cada um e a partir disso escolhendo os E indivíduos da elite. Em seguida, é elaborada uma nova geração composta por essa elite, M novos indivíduos gerados por mutação e C outros gerados do cruzamento enviesado. Dessa forma, são construídas novas gerações até que seja atingida uma condição de parada, que nesse caso determinamos como o máximo de G gerações.

Nesse projeto, cada chave aleatória nos cromossomos foi associada a um item a ser alocado (ou seja, o total m de chaves num cromossomo é o número total de itens da instância, incluindo as cópias). Este vetor de chaves foi ordenado de forma crescente, gerando uma ordenação para os itens. Então, a heurística BL foi aplicada usando os itens da ordenação dada por cada indivíduo para alocar os itens na faixa, resultando em um comprimento final utilizado como parâmetro f do indivíduo.

Para exemplificar como o *fitness* f é calculado para cada indivíduo, considere uma instância cujos itens são um triângulo retângulo (item A), um quadrado (item B) e um triângulo equilátero (item C), como na Figura 11. Considere também que foi criado um indivíduo 1, em que foi gerada uma chave aleatória para cada um desses itens, presentes na mesma figura. Podemos ver então, na Figura 12a como fica a sequência do cromossomo de 1 após a ordenação dos itens e chaves em ordem crescente, e em 12b sua alocação por meio da BL baseada nessa ordem. O comprimento usado nesta alocação é o valor de f para o indivíduo 1. O mesmo vale para um indivíduo 2, representado pelas chaves na Figura 13a e sua ordenação em 13b.

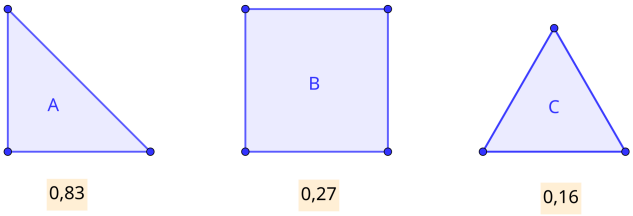
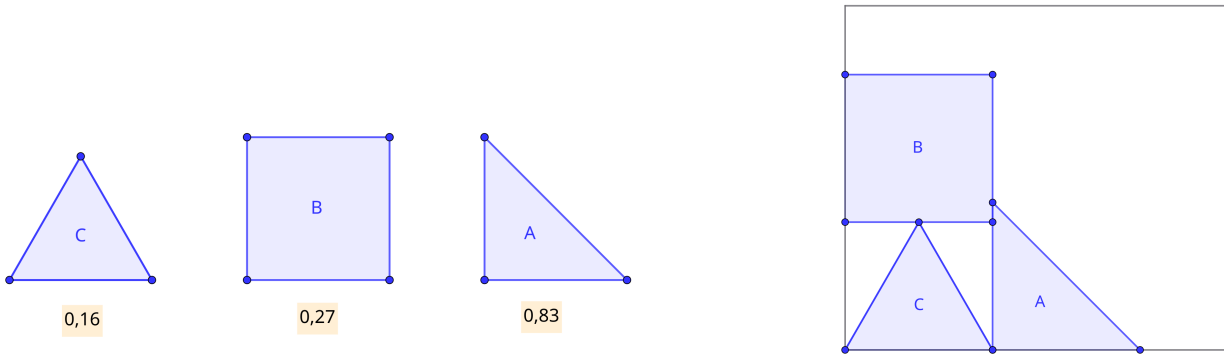
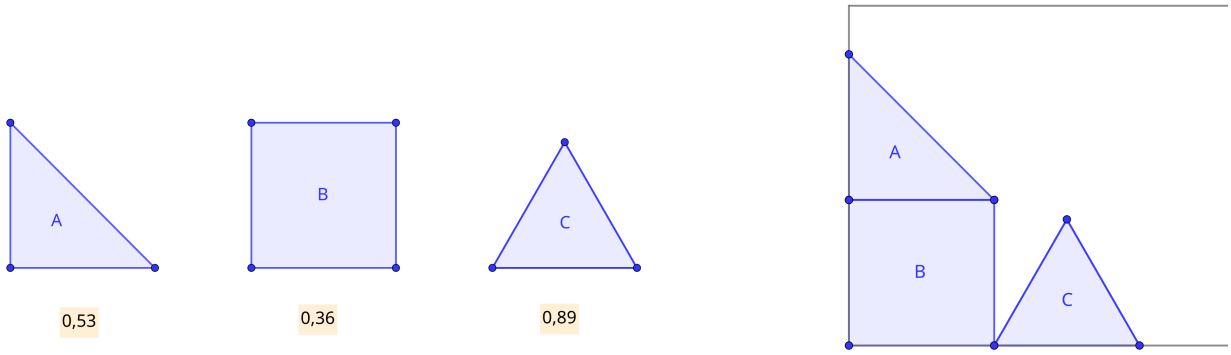


Figura 11: Chaves aleatórias geradas para o primeiro indivíduo.



(a) (b)

Figura 12: Indivíduo 1 ordenado conforme as chaves em ordem crescente em (a) e e sua alocação com base na *BL* seguindo tal ordenação em (b).



(a) (b)

Figura 13: Chaves do segundo indivíduo geradas em (a) e então sua alocação seguindo a *BL* após ordenar tais chaves, em (b).

Para exemplificar o cruzamento, usando o mesmo exemplo, o cromossomo de cada um dos pais é tomado em sua ordem original (ou seja, antes da ordenação de chaves) e para cada item do indivíduo novo é sorteado de qual indivíduo será proveniente a chave atrelada. Observe na Figura 14 o descendente 3 criado a partir do cruzamento entre 1 e 2 e em 15 como fica então sua ordenação. Perceba que, embora os valores não sejam os mesmos, sua ordenação é a mesma do indivíduo 2, ou seja, nem sempre o cruzamento gera indivíduos diferentes.

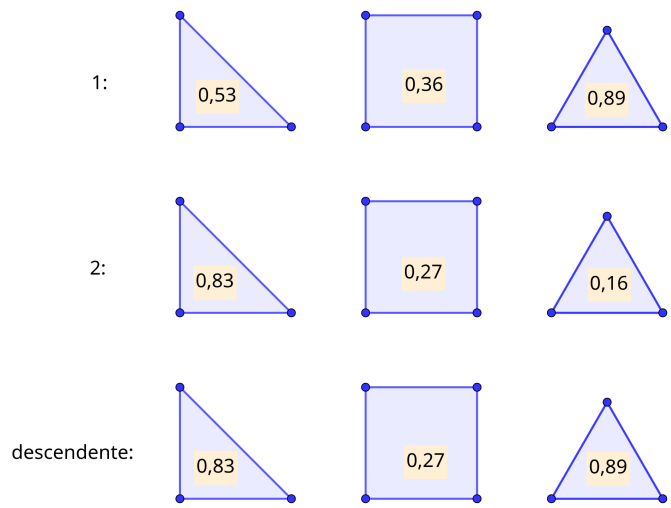


Figura 14: Cruzamento entre indivíduos 1 e 2 gerando um descendente 3.

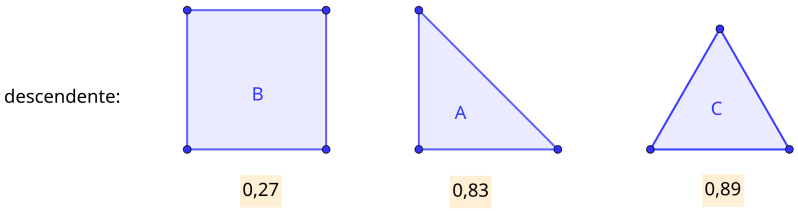


Figura 15: Ordenação final do descendente 3, com chaves ordenadas em ordem crescente.

Implementamos a *BRKGA* em linguagem C, com as mesmas especificações de leitura feitas para a implementação da *BL*, especificadas na Seção 3.Os valores dos parâmetros I, G, E, C e b são passados para o programa no momento de sua execução. A estrutura de dados dos itens e *NFPs* também é a mesma apresentada na implementação da *BL*.

Em relação à estrutura de dados específica para o algoritmo genético, temos a população que armazena os valores referentes à quantidade de indivíduos E da elite, M da mutação e C do cruzamento, o viés b do cruzamento, G da quantidade de gerações totais, I total de indivíduos na população, além de um ponteiro para o vetor fixo de indivíduos ao longo das gerações e um ponteiro para um vetor "lixo" de indivíduos, utilizado para armazenar os indivíduos provenientes do cruzamento antes de colocá-los no vetor fixo.

Cada indivíduo é composto pelo seu *fitness*, um ponteiro para um vetor que armazena as chaves na ordem original de criação e um ponteiro para vetor dos itens, em que cada um armazena o número x de um item e a chave k associada a ele.

Após a leitura dos dados, o programa passa pelas etapas a seguir:

1. Constrói a primeira geração de I indivíduos, sendo que 4 indivíduos tem suas chaves selecionadas a partir das melhores ordenações para a *BL* (aquelas citadas na Seção 3.1, dadas pela BL_{31}, BL_{21}, BL_{52} e BL_{11}), enquanto os outros $I - 4$ tem as chaves geradas aleatoriamente;
2. Para cada indivíduo i, $i = 1, \ldots, I$:
   (a) Ordena o cromossomo de i;
   (b) Chama a *BL*, obtendo o *fitness* f de i;
3. Para cada geração g, $g = 1, \ldots, G$:
   (a) Ordena os E melhores indivíduos a partir do valor do *fitness* na elite;

(b) Gera C indivíduos a partir do cruzamento de indivíduos elite e não-elite;

(c) Mantém a elite da geração $g-1$ e substitui os não-elite pelos C advindos do cruzamento;

(d) Completa a população com M indivíduos novos gerados por mutação, com chaves aleatórias;

(e) Para cada indivíduo i, $i = 1, \ldots, (C+M) = 1, \ldots, (I-E)$:

i. Ordena o cromossomo de i;

ii. Aplica a BL para calcular o *fitness* de i;

Por fim o programa identifica qual o indivíduo com melhor *fitness* resultante e imprime em um arquivo Latex a partir da ferramenta *Tikz* o leiaute correspondente, em que o recipiente é representado em azul, o comprimento obtido é a linha vertical pontilhada e os itens são coloridos (itens não convexos são representados por suas partes convexas conectadas, de mesma cor). Imprime também as informações de comprimento mínimo alcançado e tempo de execução num único arquivo de texto para facilitar sua disposição em forma de tabela (como a da Seção 3.1).

4.1 Experimentos numéricos

A meta-heurística *BRKGA* utilizou como parâmetros $I = 100$, $E = 20$, $M = 10$, $C = 70$ e $G = 10$. Para cada instância executamos o *BRKGA* duas vezes, uma com $b = 0,7$ e outra com $b = 0,5$. Os experimentos foram feitas usando as mesmas vinte e nove instâncias clássicas da literatura[2] da Seção 3.1, executadas também no mesmo computador presente nesta.

Os resultados obtidos se encontram na Tabela 3. A coluna "Instância" contém o nome de cada instância utilizada, a "Itens" retrata o número de itens da instância (incluindo as cópias de cada item). Em seguida, os grupos de duas colunas apresentam os resultado de cada execução da *BRKGA*, uma com viés $0,5$ e a outra com viés $0,7$. A coluna "Comp" representa o comprimento obtido para a instância pela *BRKGA* correspondente, em que o valor em destaque em cada linha é o menor resultado obtido e a coluna "T(s)" apresenta o tempo em segundos que a heurística levou para resolver cada instância.

Tabela 3: Resultados obtidos pelas duas variações da meta-heurística *BRKGA*, com viés de $0,5$ e $0,7$.

		BRKGA - viés $0,5$		*BRKGA* - viés $0,7$	
Instância	Itens	Comp	T (s)	Comp	T (s)
albano	24	10768,74	674,09	10878,59	637,40
blaze1	28	28,67	405,20	27,87	402,13
blaze2	16	21,23	86,83	21,13	87,11
dagli	30	66,12	597,12	65,17	655,30
dighe1	16	133,72	6,79	133,02	6,69
dighe2	10	129,70	2,33	132,87	2,31
fu	12	35,97	1,85	35,14	1,69
jakobs1	25	12,00	108,46	12,00	111,80
jakobs2	25	28,00	220,78	27,50	216,67
mao	20	2031,75	977,08	2032,92	982,93
marques	24	84,75	477,07	83,00	483,63
poly1A	15	16,57	19,08	16,51	18,22
poly2A	30	31,90	163,26	31,96	165,36
poly2b	30	34,25	201,48	33,96	199,96
poly3A	45	47,06	573,20	47,15	580,47
poly3b	45	45,29	691,01	44,29	681,84
poly4A	60	62,36	1353,01	62,49	1381,14
poly4b	60	55,97	1490,48	55,79	1497,29
poly5A	75	77,10	2624,35	77,57	2655,93
poly5b	75	67,22	2874,35	67,07	2847,81
shapes0	43	64,75	9090,53	64,00	9049,48
shapes1	43	65,00	9094,61	65,00	9295,66
shirts	99	65,37	15331,76	64,77	14867,72
three	3	6,20	0,04	6,20	0,05
threep2	6	9,33	0,33	9,33	0,31
threep2w9	6	8,50	0,31	8,50	0,32
threep3	9	14,00	1,04	13,53	1,03
threep3w9	9	11,25	1,09	11,10	0,97
trousers	64	269,33	3789,94	271,47	3627,32

[2] `https://www.euro-online.org/websites/esicup/data-sets/`, acessado em 14/08/2025.

Os resultados nos levam a acreditar que o viés de $0,7$ aplicado no cruzamento entre indivíduos, ou seja, enviesar o resultado para aumentar a probabilidade de que cada chave do novo cromossomo proceda do indivíduo elite escolhido, obtém melhores soluções, ao menos para as instâncias apresentadas.

5 Comparação entre métodos utilizados

A seguir, na Tabela 4 temos a comparação entre os dois resultados da *BRKGA* e os quatro melhores da *BL*. A coluna "Instância" contém o nome de cada instância utilizada, a "Itens" retrata o número de itens da instância (incluindo as cópias de cada item). Em seguida, os grupos de duas colunas apresentam os resultado de cada execução com o respectivo método utilizado no topo da coluna. A coluna "Comp" representa o comprimento obtido para a instância pela heurística correspondente, em que o valor em destaque em cada linha é o menor resultado obtido e a coluna "T(s)" apresenta o tempo em segundos que a heurística levou para resolver cada instância.

É possível perceber que em todas as instâncias o comprimento final obtida pela *BRKGA* foi melhor que todas da *BL* apresentadas, incluindo as outras oito variações que se encontram nas tabelas 1 e 2. O único caso em que o resultado da *BRKGA* é igual a algum da *BL* é a instância *Three*, cuja solução $6,20$ já tinha sido obtida pela BL_{12}. Ainda assim, em muitos casos essa diferença é muito pequena e, ao levar em conta o fato de que o custo computacional e de tempo da *BRKGA* é em média 1500 vezes maior que os da *BL*, não é necessariamente recomendado seu uso.

Tabela 4: Resultados obtidos pelas duas variações da $BRKGA$ e os quatro melhores da BL.

		$BRKGA$ - bias 0.5		$BRKGA$ - bias 0.7		BL_{31}		BL_{21}		BL_{52}		BL_{11}	
Instância	Itens	Comp	T (s)	Comp	T (s)	Comp	T (s)	Comp	T (s)	Comp	T (s)	Comp	T (s)
albano	24	10768.74	674.09	10878.59	637.40	11376.46	0.42	11818.58	0.29	13525.12	0.36	11818.58	0.37
blaze1	28	28.67	405.20	27.87	402.13	28.67	0.22	31.20	0.20	34.34	0.21	30.35	0.28
blaze2	16	21.23	86.83	21.13	87.11	21.50	0.08	22.10	0.06	25.29	0.04	21.50	0.09
dagli	30	66.12	597.12	65.17	655.30	75.47	0.57	73.33	0.39	89.92	0.23	67.69	0.43
dighe1	16	133.72	6.79	133.02	6.69	146.96	0.00	167.06	0.00	165.75	0.00	163.26	0.00
dighe2	10	129.70	2.33	132.87	2.31	145.56	0.00	163.23	0.00	165.16	0.00	154.30	0.00
fu	12	35.97	1.85	35.14	1.69	42.25	0.00	36.46	0.00	43.46	0.00	42.00	0.00
jakobs1	25	12.00	108.46	12.00	111.80	14.00	0.10	13.00	0.08	17.00	0.06	13.00	0.09
jakobs2	25	28.00	220.78	27.50	216.67	30.00	0.21	30.00	0.17	32.00	0.17	29.60	0.16
mao	20	2031.75	977.08	2032.92	982.93	2119.63	0.53	2289.22	0.46	2509.27	0.46	2305.08	0.46
marques	24	84.75	477.07	83.00	483.63	89.60	0.32	87.00	0.28	96.00	0.26	93.10	0.33
poly1A	15	16.57	19.07	16.51	18.22	18.10	0.01	20.18	0.01	22.56	0.01	22.05	0.01
poly2A	30	31.94	163.26	31.96	165.36	33.33	0.10	32.31	0.09	40.14	0.09	36.11	0.11
poly2b	30	34.25	201.48	33.96	199.96	35.25	0.12	35.96	0.11	41.01	0.11	37.07	0.13
poly3A	45	47.06	573.20	47.15	580.47	50.08	0.39	52.28	0.35	56.83	0.31	53.63	0.38
poly3b	45	45.29	691.00	44.29	681.84	45.68	0.45	45.29	0.35	51.14	0.34	48.42	0.46
poly4A	60	62.36	1353.01	62.49	1381.14	66.52	0.86	66.96	0.82	68.35	0.73	69.11	0.95
poly4b	60	55.97	1490.47	55.79	1497.29	55.97	0.91	56.99	0.85	61.77	0.82	60.25	0.96
poly5A	75	77.10	2624.35	77.57	2655.93	80.06	1.74	80.39	1.55	85.44	1.48	82.98	1.95
poly5b	75	67.22	2874.35	67.07	2847.80	69.28	1.81	67.52	1.50	70.99	1.52	71.56	1.85
shapes0	43	64.75	9090.53	64.00	9049.48	67.00	7.26	68.50	5.92	66.00	4.59	68.50	6.04
shapes1	43	65.00	9094.61	65.00	9295.66	67.00	6.83	68.50	5.89	66.00	4.60	68.50	5.68
shirts	99	65.37	15331.76	64.77	14867.72	68.56	7.76	68.04	7.71	67.05	7.96	68.04	7.63
three	3	6.20	0.04	6.20	0.05	7.00	0.00	7.00	0.00	7.00	0.00	6.67	0.00
threep2	6	9.33	0.33	9.33	0.31	10.00	0.00	10.00	0.00	10.00	0.00	10.67	0.00
threep2w9	6	8.50	0.31	8.50	0.32	9.27	0.00	9.27	0.00	9.27	0.00	9.67	0.00
threep3	9	14.00	1.04	13.53	1.03	16.33	0.00	16.33	0.00	16.33	0.00	16.33	0.00
threep3w9	9	11.25	1.09	11.10	0.97	14.00	0.00	14.00	0.00	14.00	0.00	12.17	0.00
trousers	64	269.33	3789.94	271.47	3627.32	281.21	1.63	283.60	1.98	305.89	2.01	283.60	1.90

Observe nas figuras 16, 17, 18, 19 e 20 os leiautes obtidos na melhor e na pior alocação para a instância *fu* pela *BL*, a alocação desta obtida pela *BRKGA*, a melhor alocação obtida pela *BL* para a instância *blaze1* e a alocação desta pela *BRKGA*, com os respectivos comprimentos finais.

O exemplo utilizado na Seção 2 ao explicar o problema, a Figura 1, é o pior leiaute obtido para a instância *blaze1*, com comprimento final 36,92 e ordenação dos itens baseada na heurística BL_{32} (ordenados de forma crescente pelo comprimento de cada item).

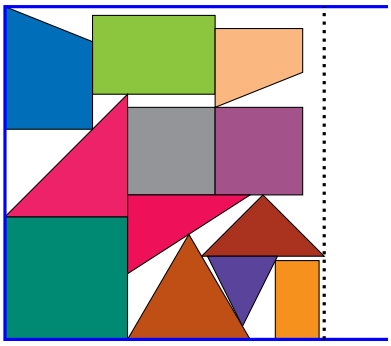


Figura 16: Melhor resultado da instância *fu* pela BL (comprimento 36,46), obtido pela BL_{21}.

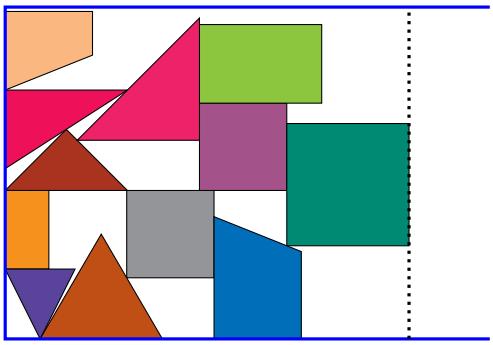


Figura 17: Pior resultado da instância *fu* (comprimento 46,25), obtido pela BL_{12}.

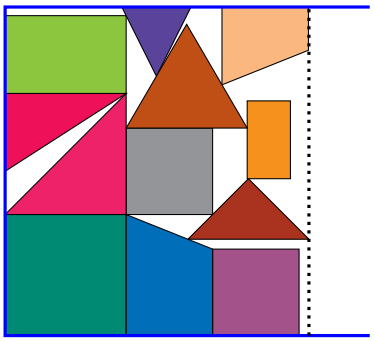


Figura 18: Melhor resultado da instância *fu* (comprimento 35,14), obtido pela $BRKGA$.

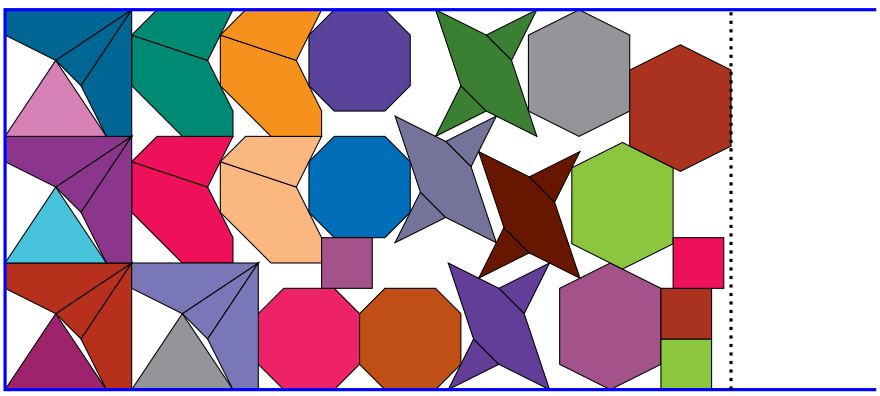


Figura 19: Melhor resultado da instância *blaze1* pela BL (comprimento 28,67), obtido pela BL_{31}.

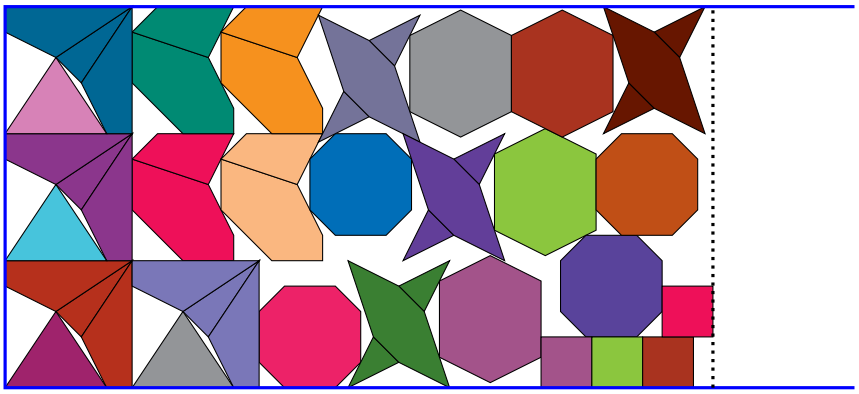


Figura 20: Melhor resultado da instância *blaze1* (comprimento 27,87), obtido pela *BRKGA*.

6 Considerações Finais

Este trabalho é a continuação de um Projeto de Iniciação Científica que consistiu em estudar otimização linear, nos casos contínuo e inteiro, focando na modelagem. Estudamos a partir de [3] a modelagem de um problema de corte bidimensional regular, em que os itens são retângulos, para então entender como seria feita a resolução de um problema de corte de itens irregulares (polígonos convexos e não convexos) em faixa.

No projeto atual, para resolução de tal problema, foi necessário estudar técnicas para verificar a sobreposição de itens (particularmente, o *No-Fit Polygon*) e dois métodos para resolução do problema, implementadas em linguagem C, a heurística construtiva *Bottom-Left* e a meta-heurística *BRKGA*.

Foram utilizadas doze combinações diferentes de ordenação dos itens para então implementar a *BL* e experimentos numéricos foram realizados com um conjunto de instâncias clássicas da literatura. Estes experimentos mostraram que a heurística foi capaz de encontrar soluções boas e ruins dependendo da ordenação, sendo as BL_{31} e BL_{21} as duas que obtiveram melhores resultados, porém todas em um tempo computacional muito pequeno.

Foram utilizados também dois viés diferentes para a implementação da *BRKGA* nessas mesmas instâncias, cujos experimentos mostraram melhores soluções, como o esperado, porém com um gasto computacional muito alto, dificultando seu uso. Devido a isso, uma possível continuação desse trabalho é a tentativa de melhorar o código, além de implementar a rotação dos itens e realizar mais testes com outras instâncias, tentando obter melhores resultados da *BRKGA*.

Referências

[1] Julia Bennell e José Oliveira. "The geometry of nesting problems: A tutorial". Em: *European Journal of Operational Research* 184.2 (2008), pp. 397–415. URL: https://www.sciencedirect.com/science/article/pii/S0377221706012379.

[2] Julia Bennell e José Oliveira. "A tutorial in irregular shape packing problems". Em: *Journal of the Operational Research Society* 60.sup1 (2009), S93–S105. URL: https://doi.org/10.1057/jors.2008.169.

[3] Horacio Yanasse et al. *Pesquisa Operacional: Para cursos de engenharia*. Rio de Janeiro: Elsevier Brasil, 2015. ISBN: 9788535281835. URL: https://books.google.com.br/books?id=aZbpCgAAQBAJ.

[4] Leandro Mundim, Marina Andretta e Thiago Alves. "A biased random key genetic algorithm for open dimension nesting problems using no-fit raster". Em: *Expert Systems with Applications* 81 (2017), pp. 358–371. ISSN: 0957-4174. URL: https://www.sciencedirect.com/science/article/pii/S0957417417302233.

[5] Leandro Mundim et al. "A general heuristic for two-dimensional nesting problems with limited-size containers". Em: *International Journal of Production Research* 56.1-2 (2018), pp. 709–732. URL: https://doi.org/10.1080/00207543.2017.1394598.